

DWM EXP2:- Implementation of OLAP operations

Aim:

The aim of this experiment is to implement OLAP (Online Analytical Processing) operations including **Slice**, **Dice**, **Rollup**, **Drilldown**, and **Pivot** on a multidimensional dataset to better understand and manipulate data for insightful analysis.

Theory:

OLAP operations are used to analyze large volumes of data stored in a multidimensional format, typically represented as a data cube. These operations help to summarize, explore, and view data from different perspectives. The following OLAP operations are commonly used:

1. Slice:

- **Description:** The Slice operation allows you to select a single layer (or slice) of the data cube along one dimension, reducing the data to a lower-dimensional view.
- **Functionality:** It provides a way to view the data from one perspective, typically focusing on one particular value of a dimension.
- **Example:** If you have a cube with dimensions (Year, Product, Region), performing a slice on the **Year** dimension could give you the data for just one year (e.g., 2020), allowing analysis on that year's data across products and regions.

2. Dice:

- **Description:** The Dice operation is an extension of the Slice operation, where data is filtered along multiple dimensions to create a subcube.
- **Functionality:** It selects specific values for multiple dimensions, providing a multidimensional slice of the data.
- **Example:** You could dice the cube by selecting sales data for specific years, specific products, and specific regions, narrowing down the data to only a subset of interest (e.g., sales for "Product A" in "North" region for the years 2019 and 2020).

3. Rollup:

- **Description:** Rollup summarizes data along a dimension by aggregating the data into

higher-level hierarchies.

- **Functionality:** This operation allows for summarization of the data, typically to higher levels of granularity, like moving from daily data to monthly or yearly data.
- **Example:** You could roll up data from individual months into a yearly view, aggregating sales data by year instead of by month.

4. Drilldown:

- **Description:** Drilldown is the opposite of Rollup. It allows you to view data at a more granular level, drilling down into more detailed data.
- **Functionality:** This operation enables you to explore lower levels of data to gain deeper insights.
- **Example:** You could drill down on yearly data to view the data by month or even day, providing more detailed information.

5. Pivot:

- **Description:** The Pivot operation changes the orientation of the data in the data cube, allowing you to view it from different perspectives.
- **Functionality:** It rearranges the dimensions of the data, usually swapping rows and columns, to offer a clearer view of the data.
- **Example:** You might pivot the sales data so that the columns show products and the rows show regions, allowing for easier comparison of sales across different regions for each product.

Program Output:

The output of implementing these OLAP operations would be in the form of a transformed dataset. Each operation would produce a new table or view, showing the data in the desired format.

- **Slice:** Would produce a reduced dataset focusing on a single dimension's value.
- **Dice:** Would show a smaller, multidimensional subset of the data.
- **Rollup:** Would provide an aggregated, summarized view of the data at a higher level.
- **Drilldown:** Would show more detailed, lower-level data.
- **Pivot:** Would present the data from a different orientation, making comparisons easier.

Review Questions:

1. What is the difference between the Slice and Dice operations in OLAP?

- **Slice** involves selecting a specific value from one dimension of the data, reducing the dimensionality of the data to focus on one particular slice of the cube.
- **Dice**, on the other hand, selects specific values from multiple dimensions, creating a subcube of the data, and allowing you to look at a particular section of the multidimensional data.

2. How does the Roll-up operation help in summarizing large volumes of data?

- **Roll-up** aggregates the data along a higher level of hierarchy (e.g., summarizing sales data by year instead of by month). It reduces the amount of detail by grouping data, making it easier to identify trends and patterns across larger datasets.

3. Give an example of a scenario where Pivoting the data provides a clearer insight than traditional tabular view?

- **Example:** Consider a sales dataset where each row contains sales data with columns for the product, region, and date. If you pivot the data so that the rows represent regions and the columns represent products, you can quickly compare sales performance across different regions for each product, which may not be as easily understood in a traditional tabular format with sales data scattered across multiple rows and columns.

Conclusion:

- The **Slice and Dice** operations allow users to filter data along one or more dimensions, respectively, enabling more focused analysis.
- **Roll-up** and **Drilldown** operations allow users to either aggregate data at a higher level or break it down to a finer level of detail, making it easier to understand trends and relationships in large datasets.
- **Pivot** allows the reorganization of data for a clearer perspective, enhancing the ability to compare data across multiple dimensions.

GITBUB: <https://github.com/panchaldeep1123/dwm>

My Session



Actions ▾



Reset Session



Save

Sort

most recent last

View

Statements and Results

Include

All Statements

Statement 1



```
CREATE TABLE Time_dw (  
  time_id INT PRIMARY KEY,  
  quarter INT,  
  day INT ,  
  month char(40),  
  year INT  
)  
  
CREATE TABLE Branch (  
  branch_id INT PRIMARY KEY,  
  name char(50),  
  state char(50),  
  country char(50)  
)  
  
CREATE TABLE Location (  
  location_id INT PRIMARY KEY,  
  CITY CHAR(50),  
  state char(50),  
  country char(50)  
)  
  
CREATE TABLE Product (  
  product_id INT PRIMARY KEY,  
  quantity INT,  
  price INT,  
  model_number INT,  
  type VARCHAR(50),  
  name VARCHAR(100)  
)
```

```
create table Sales_Fact (  
sales_id INT PRIMARY KEY,  
time_id INT,  
product_id INT,  
branch_id INT,  
location_id INT,  
quantity INT,  
sales_amount DECIMAL(10,2),  
revenue DECIMAL(10,2),  
FOREIGN KEY (time_id) REFERENCES Time_dw(time_id),  
FOREIGN KEY (product_id) REFERENCES Product(product_id),  
FOREIGN KEY (branch_id) REFERENCES Branch(branch_id),  
FOREIGN KEY (location_id) REFERENCES Location(location_id)  
)
```

ORA-00922: missing or invalid option

More Details: <https://docs.oracle.com/error-help/db/ora-00922>

Statement **2**



```
CREATE TABLE Time_dw (  
time_id INT PRIMARY KEY,  
quarter INT,  
day INT ,  
month char(40),  
year INT  
)
```

Table created.

Statement **3**



```
CREATE TABLE Branch (  
branch_id INT PRIMARY KEY,  
name char(50),  
state char(50),  
country char(50)  
)
```

Table created.

Statement **4**



```
CREATE TABLE Location (  
  location_id INT PRIMARY KEY,  
  CITY CHAR(50),  
  state char(50),  
  country char(50)  
)
```

Table created.

Statement **5**



```
CREATE TABLE Product (  
  product_id INT PRIMARY KEY,  
  quantity INT,  
  price INT,  
  model_number INT,  
  type VARCHAR(50),  
  name VARCHAR(100)  
)
```

Table created.

Statement **6**



```
create table Sales_Fact (  
  sales_id INT PRIMARY KEY,  
  time_id INT,  
  product_id INT,  
  branch_id INT,  
  location_id INT,  
  quantity INT,  
  sales_amount DECIMAL(10,2),  
  revenue DECIMAL(10,2),  
  FOREIGN KEY (time_id) REFERENCES Time_dw(time_id),  
  FOREIGN KEY (product_id) REFERENCES Product(product_id),  
  FOREIGN KEY (branch_id) REFERENCES Branch(branch_id),  
  FOREIGN KEY (location_id) REFERENCES Location(location_id)  
)
```

Table created.

Statement **7**



```
INSERT INTO Time_dw VALUES (1, 1, 5, 'January', 2025)
```

1 row(s) inserted.

Statement **8**



```
INSERT INTO Time_dw VALUES (2, 2, 7, 'May', 2025)
```

1 row(s) inserted.

Statement **9**



```
INSERT INTO Time_dw VALUES (3, 1, 18, 'March', 2025)
```

1 row(s) inserted.

Statement **10**



```
INSERT INTO Time_dw VALUES (4, 1, 14, 'February', 2025)
```

1 row(s) inserted.

Statement **11**



```
INSERT INTO Time_dw VALUES (5 , 2, 1, 'June', 2025)
```

1 row(s) inserted.

Statement **12**



```
INSERT INTO product VALUES (1, 10, 80000, 105, 'Electronics - Gadgets', 'Tablet')
```

1 row(s) inserted.

Statement **13**



```
INSERT INTO product VALUES (2, 12, 1700, 105, 'Electronics - wearables', 'smartwatch')
```

1 row(s) inserted.

Statement **14**



```
INSERT INTO product VALUES (3, 16, 12000, 105, 'Electronics - Gadgets', 'Tablet')
```

1 row(s) inserted.

Statement **15**



```
INSERT INTO product VALUES (4, 18, 2000, 105, 'Electronics - accesories', 'Airdopes')
```

1 row(s) inserted.

Statement **16**



```
INSERT INTO product VALUES (5, 11, 45000, 105, 'Electronics - appliances', 'TV')
```

1 row(s) inserted.

Statement **17**



```
INSERT INTO Branch VALUES (5, 'Branch E', 'Maharashtra', 'India')
```

1 row(s) inserted.

Statement **18**



```
INSERT INTO Branch VALUES (6, 'Branch F', 'Delhi', 'India')
```

1 row(s) inserted.

Statement **19**



```
INSERT INTO Branch VALUES (7, 'Branch G', 'Karnataka', 'India')
```

1 row(s) inserted.

Statement **20**



```
INSERT INTO Branch VALUES (8, 'Branch H', 'West Bengal', 'India')
```

1 row(s) inserted.

Statement **21**



```
INSERT INTO Branch VALUES (9, 'Branch I', 'Tamil Nadu', 'India')
```

1 row(s) inserted.

Statement **22**



```
INSERT INTO Location VALUES (1, 'Mumbai', 'Maharashtra', 'India')
```


1 row(s) inserted.

Statement **23**



```
INSERT INTO Location VALUES (2, 'Delhi', 'Delhi', 'India')
```

1 row(s) inserted.

Statement **24**



```
INSERT INTO Location VALUES (3, 'Bangalore', 'Karnataka', 'India')
```

1 row(s) inserted.

Statement **25**



```
INSERT INTO Location VALUES (4, 'Kolkata', 'West Bengal', 'India')
```

1 row(s) inserted.

Statement **26**



```
INSERT INTO Location VALUES (5, 'Chennai', 'Tamil Nadu', 'India')
```

1 row(s) inserted.

Statement **27**



```
ALTER TABLE Sales_Fact DROP COLUMN sales_id
```

Table altered.

Statement **28**



```
ALTER TABLE Sales_Fact  
ADD (sales_id NUMBER GENERATED ALWAYS AS IDENTITY  
(START WITH 1 INCREMENT BY 1))
```

Table altered.

Statement **29**



```
INSERT INTO Location VALUES (6, 'Chennai', 'Tamil Nadu', 'India')
```

1 row(s) inserted.

Statement **30**

```
ALTER TABLE Sales_Fact
ADD (sales_id NUMBER GENERATED ALWAYS AS IDENTITY
(START WITH 1 INCREMENT BY 1))
```

ORA-01430: column being added already exists in table

More Details: <https://docs.oracle.com/error-help/db/ora-01430>

Statement **31**

```
ALTER TABLE Sales_Fact
ADD (sales_id NUMBER GENERATED ALWAYS AS IDENTITY
(START WITH 1 INCREMENT BY 1))
```

ORA-01430: column being added already exists in table

More Details: <https://docs.oracle.com/error-help/db/ora-01430>

Statement **32**

```
INSERT INTO Sales_Fact (
time_id,
product_id,
branch_id,
location_id,
quantity,
sales_amount,
revenue
)
SELECT
t.time_id,
p.product_id,
b.branch_id,
l.location_id,
FLOOR(DBMS_RANDOM.VALUE(1, 100)), -- Random quantity between 1 and 100
(FLOOR(DBMS_RANDOM.VALUE(1, 100)) * p.price), -- Random sales amount
(FLOOR(DBMS_RANDOM.VALUE(1, 100)) * p.price * 0.8) -- Random revenue
FROM
Time_dw t
CROSS JOIN
Product p
```

CROSS JOIN
Branch b
CROSS JOIN
Location l

750 row(s) inserted.

Statement 33



```
SELECT *  
FROM Sales_Fact  
WHERE time_id = '3'
```

TIME_ID	PRODUCT_ID	BRANCH_ID	LOCATION_ID	QUANTITY	SALES_AMOUNT	REVENUE
3	1	5	1	84	640000	588800
3	1	5	2	18	4400000	172800
3	1	5	3	67	2960000	108800
3	1	5	4	22	3200000	326400
3	1	5	5	10	4480000	403200
3	1	5	6	97	3040000	492800
3	1	6	1	56	4080000	275200
3	1	6	2	37	3440000	64000
3	1	6	3	8	2400000	364800
3	1	6	4	60	6400000	307200
3	1	6	5	70	4400000	166400
3	1	6	6	55	7040000	236800
3	1	7	1	75	2960000	192000
3	1	7	2	52	6080000	294400
3	1	7	3	13	2560000	102400

3	1	7	4	28	4000000	544000
3	1	7	5	58	7600000	588800
3	1	7	6	50	3520000	416000
3	1	8	1	65	7360000	236800
3	1	8	2	49	6240000	224000
3	1	8	3	74	2960000	588800
3	1	8	4	43	6160000	179200
3	1	8	5	88	6720000	588800
3	1	8	6	92	4000000	896000
3	1	9	1	24	5440000	531200
3	1	9	2	59	5200000	588800
3	1	9	3	85	5280000	160000
3	1	9	4	49	880000	153600
3	1	9	5	13	2880000	307200
3	1	9	6	63	400000	422400
3	2	5	1	6	37400	121040
3	2	5	2	31	149600	91120
3	2	5	3	43	85000	96560
3	2	5	4	50	64600	130560
3	2	5	5	81	163200	72080
3	2	5	6	99	74800	14960
3	2	6	1	31	3400	48960
3	2	6	2	56	81600	19040

3	2	6	3	7	146200	34000
3	2	6	4	32	73100	69360
3	2	6	5	8	88400	47600
3	2	6	6	69	22100	119680
3	2	7	1	38	6800	12240
3	2	7	2	92	107100	55760
3	2	7	3	70	62900	13600
3	2	7	4	94	124100	44880
3	2	7	5	74	35700	36720
3	2	7	6	17	15300	63920
3	2	8	1	59	22100	10880
3	2	8	2	15	107100	43520

Download CSV

Rows 1 - 50. More rows exist.

Statement 34



```
SELECT *  
FROM Sales_Fact  
WHERE time_id = '2'  
AND branch_id = 7
```

TIME_ID	PRODUCT_ID	BRANCH_ID	LOCATION_ID	QUANTITY	SALES_AMOUNT	REVENUE
2	1	7	1	17	6000000	403200
2	1	7	2	81	160000	390400
2	1	7	3	66	7840000	377600
2	1	7	4	69	5520000	390400

2	1	7	5	85	5520000	396800
2	1	7	6	83	5680000	601600
2	2	7	1	26	153000	107440
2	2	7	2	97	25500	108800
2	2	7	3	3	100300	95200
2	2	7	4	57	163200	13600
2	2	7	5	35	159800	106080
2	2	7	6	31	13600	76160
2	3	7	1	40	144000	105600
2	3	7	2	15	408000	172800
2	3	7	3	7	24000	633600
2	3	7	4	32	648000	643200
2	3	7	5	78	1020000	748800
2	3	7	6	63	1128000	518400
2	4	7	1	39	180000	38400
2	4	7	2	21	136000	54400
2	4	7	3	68	198000	1600
2	4	7	4	23	28000	33600
2	4	7	5	29	32000	129600
2	4	7	6	78	114000	36800
2	5	7	1	83	450000	172800
2	5	7	2	44	1485000	252000
2	5	7	3	23	270000	342000

2	5	7	4	44	3375000	1368000
2	5	7	5	18	2655000	1044000
2	5	7	6	59	2745000	3168000

Download CSV

30 rows selected.

Statement 35



```
SELECT branch_id, location_id,
SUM(sales_amount) AS total_sales,
SUM(revenue) AS total_revenue
FROM Sales_Fact
```

ORA-00937: not a single-group group function

More Details: <https://docs.oracle.com/error-help/db/ora-00937>

Statement 36



```
SELECT location_id,
SUM(sales_amount) AS location_sales,
SUM(revenue) AS location_revenue
FROM Sales_Fact
WHERE branch_id = 7
GROUP BY location_id
```

LOCATION_ID	LOCATION_SALES	LOCATION_REVENUE
6	36458100	30094960
1	36340600	27986640
2	27443900	32239120
4	40286300	23970240
5	42341000	29272800
3	38962300	26359680

Download CSV

Statement 37



```
SELECT *
FROM (
  SELECT branch_id, sales_amount
  FROM Sales_Fact
  WHERE time_id = '4'
)
PIVOT (
  SUM(sales_amount) AS total
  FOR branch_id IN (6 AS branch_6, 7 AS branch_7, 8 AS branch_8)
)
```

BRANCH_6_TOTAL	BRANCH_7_TOTAL	BRANCH_8_TOTAL
44973100	51001200	49832800

Download CSV